

Unit Testing — HelixVPN nano-detail spec (Volume 8 · §11.4.169 type 1)

Unit Testing — HelixVPN nano-detail spec (Volume 8 · §11.4.169 type 1)

Revision: 1 **Last modified:** 2026-06-26T12:00:00Z

Nano-detail expansion of [§5.1 of the Volume-8 overview](#). Unit testing is the **one and only** layer where mocks, stubs, fakes, placeholders, and TODO/FIXME are constitutionally permitted (§11.4.27(A)); every other test type (see siblings [integration.md](#), [e2e.md](#), [full-automation.md](#), [challenges.md](#), [helixqa.md](#)) exercises the real system. This document fixes the unit surface across the three HelixVPN languages (Rust data plane, Go control plane, Dart UI logic), the harness, the property-test strategy, the captured-evidence shape, the determinism contract, the acceptance gate, the paired §1.1 mutation that proves the gate is not a bluff, and concrete test skeletons. Spec-only; every unproven assumption is marked UNVERIFIED.

Table of contents

- [1. Scope — what unit testing covers in HelixVPN](#)
- [2. Harness — per-language frameworks](#)
- [3. Fixtures — real vs mocked \(the §11.4.27 boundary\)](#)
- [4. Evidence taxonomy — what a unit PASS captures](#)
- [5. Determinism — N-iteration identical evidence](#)
- [6. Acceptance gate — when UNIT blocks a release](#)
- [7. The paired §1.1 mutation \(anti-bluff proof\)](#)
- [8. Test skeletons](#)

- 9. Open decisions surfaced for QA
 - Sources verified
-

1. Scope — what unit testing covers in HelixVPN

Unit testing owns **pure, deterministic logic** where a bug is a *logic* bug — not an integration, timing, or environment bug. The discriminator is: *does this code path produce its result entirely from its inputs, with no socket, no clock, no filesystem, no real process?* If yes, it is a unit test; if it touches real infra it belongs in integration.md (§11.4.27 forbids fakes there).

Surface	Component	Unit-testable logic
Policy compiler	helix-go svc-policy (../v03-control-plane/svc-policy.md)	ACL → per-peer AllowedIPs set + edge verdict-map; deny correctness (empty/parse-failed ACL ⇒ zero routes)
IPAM allocator (logic half)	helix-go svc-ipam (../v03-control-plane/svc-ipam.md)	ULA /48 carve, host-subnet math, 4via6 mapping, collision math (D4 [SYNTHESIS])
Coordinator delta diff	helix-go svc-coordinator (../v03-control-plane/svc-coordinator.md)	snapshot→minimal-delta computation; idempotency; ordering invariance
MASQUE / HTTP-Datagram framing	helix-core transport-masque-quic (../v02-data-plane/transport-masque-quic.md)	RFC 9221 unreliable-datagram encode/decode, context-ID framing, MTU clamp 1420
Transport-selection ladder (decision logic)	helix-core transport-selection-ladder (../v02-data-plane/transport-selection-ladder.md)	escalation order, per-network memory key, handshake-failure→next-rung mapping

Surface	Component	Unit-testable logic
Client status FSM	selection-ladder.md) helix-core orchestrator-and-state (../v02-data-plane/orchestrator-and-state.md)	Disconnected→Connecting→Connected→Reconnecting→Blocked transitions; loading is a distinct state (§11.4.107(3))
Kill-switch rule synthesis	helix-core (../v04-client/helix-core-rust.md)	firewall-rule set generated for a given tunnel state (the rule is not their application)
WireGuard key/AllowedIPs validation	helix-core wireguard-core (../v02-data-plane/wireguard-core.md)	base64 key parse, AllowedIPs CIDR parse + subnet containment
UI status→view pure function	helix-ui state-management (../v04-client/state-management.md)	TunnelStatus → ConnState mapping that drives ConnectButton/ShieldIndicator

Out of scope for UNIT (delegated, never faked): RLS multi-tenant isolation (INT, real Postgres — [integration.md §1](#)); reachability through a real tunnel (E2E — [e2e.md](#)); kill-switch *leak capture* (SEC); iOS NE memory ceiling (MEM); any assertion needing a packet on a wire. Attempting these in a unit test with a mock socket is precisely the §11.4.27 violation this layer's boundary forbids — a green mock proves the mock, not the product.

2. Harness — per-language frameworks

Language	Runner	Property testing	Coverage tool	Invocation
Rust (core, edge)	cargo test	proptest	cargo llvm-cov	cargo test --workspace
Go (control plane)	go test (stdlib testing)	gopter (or testing/quick)	go test -cover	go test ./helix-go/...

Language	Runner	Property testing	Coverage tool	Invocation
Dart (UI logic)	dart test / flutter test	glados (Dart property lib) UNVERIFIED	flutter test --coverage	melos run test

The runners are wired into the local Makefile `test:` target ([overview §9](#)) and the **pre-commit** git hook (§11.4.75 Layer-1) so UNIT runs before any commit lands. Property tests are mandatory for the **policy compiler** and **IPAM allocator** — the two components where exhaustive hand-written cases predictably miss the boundary that bites (an off-by-one in the ULA carve, an ACL ordering that silently grants a route). `proptest/gopter` shrink a failing input to its minimal counter-example, which becomes a committed regression-guard fixture (§11.4.135).

UNVERIFIED: the exact Dart property-test library (`glados` vs hand-rolled `testing/quick-style` generators) is not yet pinned; the obligation (property coverage of the `status`→`view pure function`) is fixed, the library is a Phase-1 selection.

3. Fixtures — real vs mocked (the §11.4.27 boundary)

Unit fixtures are **inputs only**, never live infrastructure. The closed set:

Fixture kind	Example	Real or mock	Rationale
Golden input table	<code>acl.yaml</code> → expected AllowedIPs set	real data, committed	the compiler's regression oracle (§11.4.135)
Synthetic event log	a sequence of <code>handshake_failed</code> events fed to the ladder FSM	real data, no network	the FSM is pure; the events stand in for the wire
In-memory trait double	a <code>Clock</code> returning a fixed instant; a <code>KeyStore</code> returning a canned keypair	mock — permitted here only	isolates the logic-under-test from time/IO (§11.4.27(A))
Byte-vector fixtures	a captured MASQUE datagram for decode round-trip	real bytes, committed generated	proves framing parses a real-shaped buffer

Fixture kind	Example	Real or mock	Rationale
Property generators	proptest strategy producing arbitrary ACLs / overlay ranges		exhaustive boundary search

The mock rule, stated mechanically: a mock may stand in only for a *dependency* of the unit-under-test (a clock, a key source, an RNG seed), never for the *thing being asserted*. A test that mocks the policy compiler and asserts the mock returned the expected set is a §11.4.27 + §11.4 PASS-bluff and is caught by the analyzer self-validation discipline (§7). Mocks are also forbidden from leaking into production paths: production code must not import a mock module (the CM-NO-FAKES-BEYOND-UNIT-TESTS gate, §11.4.27(A)).

4. Evidence taxonomy — what a unit PASS captures

A unit PASS is the *cheapest, least conclusive* signal in the §11.4.108 four-layer model (SOURCE layer only) — it proves internal logic, never that the feature works on a deployed target. Its captured evidence is therefore modest but still non-trivial:

Evidence artifact	Produced by	Committed where
test-runner output (pass/fail counts, per-test names)	cargo test / go test / flutter test	qa-results/unit/<lang>_<ts>.log
coverage report (line + branch)	llvm-cov / go test -cover / --coverage	qa-results/unit/coverage/
golden-table fixture (compiler)	committed	helix-go/.../testdata/acl_golden/ (regression guard §11.4.135)
proptest/gopter shrunk counter-example	on failure	committed as a regression fixture

Per §11.4.69 the unit PASS still flows through `ab_pass_with_evidence <desc> <evidence_path>` when run inside the orchestrated suite, citing the runner log + coverage report — bare `ab_pass` is a release blocker even at the

unit layer. The unit log is **not** sufficient evidence for a *feature* closure; the coverage-ledger cell for a feature only reaches AUTONOMOUS_VERIFIED when its INT/E2E/SEC cells also carry §11.4.69 sink-side evidence ([overview §6](#)).

5. Determinism — N-iteration identical evidence

Per §11.4.50 every unit PASS must reproduce **identically** across N runs (N=3 normal, N=10 cycle-validation) against the same source tree. Unit tests are the easiest layer to make deterministic and the easiest to *accidentally* make flaky; the mandate is mechanical:

1. **No real clock, no real RNG, no real network** — all injected as fixtures (§3), so the same inputs always produce the same outputs.
2. **Seeded property tests** — proptest/gopter run with a **fixed seed** in CI so the generated case-set is identical each run; a newly-shrunk failure is committed as a fixed fixture (the seed only varies in an explicit fuzz job).
3. **No map-ordering leaks** — Go map iteration order is randomised; any test asserting an ordered result sorts first, or the diff is order-invariant by construction (also a CONC property, [overview §5.10](#)).
4. The FA wrapper `ab_run_n_times <name> 3 <fn>` (UNVERIFIED exact lib path) hashes the runner's JSON output per iteration and FAILs on any divergence — there is no "first-pass-was-a-flake" escape (§11.4.50).

A unit test that passes 9/10 is **auto-FAIL**, treated as a real defect (a hidden clock/RNG/ordering dependency), never demoted to "flake" — §11.4.50 enforces the §11.4.7 demotion-evidence rule mechanically.

6. Acceptance gate — when UNIT blocks a release

Gate	Bar	Layer
pre-commit hook (§11.4.75 Layer-1)	every staged <code>*.rs/*.go/*.dart</code> 's package unit tests GREEN	local, blocks commit
coverage floor ratchet (§11.4.50 feature-coverage matrix)	line+branch coverage $\geq 70\% \rightarrow 85\% \rightarrow$ 95% across phases	make test
default-deny floor	the policy-compiler <code>empty_acl_grants_nothing +</code>	every gate

Gate	Bar	Layer
	parse_error_is_fail_closed tests are release-blocking unit tests	
CM-NO-FAKES-BEYOND-UNIT-TESTS (§11.4.27(A))	no production path imports a mock; mocks confined to *_test.*	pre-build

UNIT is a contributor to **G5** (Phase-0: FFI round-trip drivable from Dart, [overview §7.1](#)) and underpins every AC by proving the pure-logic core of each feature, but UNIT alone **never** clears an AC — the AC gates demand E2E/SEC/CHAL captured evidence. A unit-only "green" on a feature whose E2E is red is exactly the bluff §11.4 forbids; the coverage ledger's v_coverage_gaps view ([overview §6.1](#)) keeps that feature in the release-blocker set until its higher cells verify.

7. The paired §1.1 mutation (anti-bluff proof)

Every unit gate ships a paired mutation that **must** make it FAIL — proving the gate catches a real regression and is not a tautology. For the load-bearing default-deny floor:

```
# §1.1 mutation for CM-POLICY-DEFAULT-DENY (the meta-test plants
this, asserts FAIL)
- in helix-go/.../svc-policy/compile.go, change the fail-closed
branch:
    on parse error: return CompiledPolicy{}           // correct:
zero routes
    MUTATED:      return CompiledPolicy{All: peers} // fail-
OPEN: every peer routes
- assert: empty_acl_grants_nothing / parse_error_is_fail_closed
now FAIL
- restore; assert they PASS again (working-tree quiescence
§11.4.84)
```

For the **analyzer-self-validation** discipline (§11.4.107(10)) applied to unit fixtures: any unit test that itself parses a captured artifact (e.g. a unit test of the leak-detector's pcap classifier — though the detector's *integration* lives in SEC) ships a **golden-good + golden-bad fixture pair**; stripping the discriminating clause must make golden_bad start passing → the meta-test catches the mutation. This is the mechanical defeat of bluff-class **B5** at the unit layer.

The mutation/restore cycle is serialised and the working tree verified clean before any unrelated commit (§11.4.84) so a MUTATED-marked file can never be swept into a real commit.

8. Test skeletons

8.1 Rust — policy compiler default-deny (the security floor)

```
// helix-core (or helix-go bridge) — interface under test
pub trait PolicyCompiler {
    /// Compile a tenant ACL into per-peer AllowedIPs + an edge
    /// verdict map.
    /// MUST be default-deny: an empty OR parse-failed ACL yields
    /// ZERO allowed routes.
    fn compile(&self, acl: &Acl, peers: &[PeerId]) ->
        Result<CompiledPolicy, CompileError>;
}

#[cfg(test)]
mod default_deny {
    use super::*;

    #[test] fn empty_acl_grants_nothing() { // floor: no rules => no routes
        let c = TailscaleAclCompiler::default();
        let cp = c.compile(&Acl::empty(), &[peer("alice"),
            peer("bob")]).unwrap();
        for p in cp.peers() {
            assert!(cp.allowed_ips(p).is_empty(), "fail-closed
            violated for {p:?}");
        }
    }

    #[test] fn parse_error_is_fail_closed_not_fail_open() { // never fail-OPEN on bad input
        let c = TailscaleAclCompiler::default();
        assert!(matches!(c.compile(&Acl::malformed(), &[]),
            Err(CompileError::Parse(_))));
        // the CALLER's "Err => zero routes" handling is then
        // proven in the INT layer (integration.md)
    }

    // property test: any randomly-generated ACL never grants a
    // peer a route to a
    // host outside that ACL's explicit grants (the compiler is
    // monotone in grants).
    proptest::proptest! {
        #[test] fn compiler_never_overgrants(acl in arb_acl(),
            peers in arb_peers()) {
            let cp =
                TailscaleAclCompiler::default().compile(&acl,
                    &peers).unwrap();
            for p in cp.peers() {
                for route in cp.allowed_ips(p) {

```



```

                                proptest::prop_assert!
(acl.explicitly_grants(p, route),
                                "compiler over-granted {route} to {p:?}");
                                }
                                }
                                }
                                }
                                }
}

```

8.2 Go — coordinator delta diff (order-invariance + minimality)

```

// helix-go/internal/coordinator/delta_test.go
package coordinator

import "testing"

func TestDeltaIsMinimalAndOrderInvariant(t *testing.T) {
    base := mapWith("alice@10.0.0.1", "bob@10.0.0.2")
    next := mapWith("alice@10.0.0.1", "bob@10.0.0.2",
                    "carol@10.0.0.3") // +carol only

    d := DiffNetworkMap(base, next)
    if len(d.Added) != 1 || d.Added[0].Host != "carol" {
        t.Fatalf("delta not minimal: %+v", d) // must emit ONLY
        the carol add
    }
    if len(d.Removed) != 0 {
        t.Fatalf("phantom removals: %+v", d.Removed)
    }
    // order-invariance: shuffling next's peer order yields an
    identical delta
    if got := DiffNetworkMap(base, shuffle(next)); !deltaEqual(d,
        got) {
        t.Fatalf("delta not order-invariant")
    }
}

```

8.3 Go — IPAM allocator pure address math (the logic half)

```

// helix-go/internal/ipam/carve_test.go (the persistence half is
CONC/INT)
func TestULACarveIsCollisionFreeAndInRange(t *testing.T) {
    pool := MustParseULA("fd7a:115c:a1e0::/48") //
    overlay /48 (D4 [SYNTHESIS])
    seen := map[string]bool{}
    for i := 0; i < 1000; i++ {
        ip :=
        pool.HostFor(deviceSeed(i)) // PURE:
        deterministic from seed
        if !pool.Contains(ip) { t.Fatalf("%s out of /48", ip) }
        if seen[ip.String()] { t.Fatalf("collision at %s", ip) }
    }
}

```

```

        seen[ip.String()] = true
    }
}

```

8.4 Dart — status→view pure function (UI logic, no widget)

```

// helix-ui/test/status_to_view_test.dart (pure function;
// rendering is UI/REC)
import 'package:test/test.dart';
void main() {
  test('Connecting maps to a non-green shield (loading is
        distinct, §11.4.107(3))', () {
    expect(connStateFor(TunnelStatus.connecting),
      ConnState.connecting);
    expect(shieldColorFor(ConnState.connecting),
      isNot(ShieldColor.green));
  });
  test('Blocked (kill-switch) maps to a sealed red shield', () {
    expect(shieldColorFor(connStateFor(TunnelStatus.blocked)),
      ShieldColor.red);
  });
}

```

8.5 Rust — MASQUE framing round-trip (RFC 9221, [research-masque])

```

// helix-core/crates/helix-masque/tests/framing.rs
#[test] fn http_datagram_roundtrip_preserves_payload() {
    let ctx = ContextId(2);
    let payload = b"\x00\x01\x02 inner wireguard packet bytes";
    let framed = encode_http_datagram(ctx, payload); // pure encode
    let (got_ctx, got) =
        decode_http_datagram(&framed).unwrap(); // pure decode
    assert_eq!(got_ctx, ctx);
    assert_eq!(got, payload); // byte-exact round-trip
}
#[test] fn oversize_payload_is_clamped_not_truncated_silently() {
    let big = vec![0u8; 1500]; // > overlay MTU 1420
    assert!(matches!(encode_http_datagram(ContextId(2), &big),
        Err(FrameError::TooLarge(_))));
}

```

9. Open decisions surfaced for QA

Per §11.4.66, decisions the unit layer touches but does not silently resolve:

#	Decision	Options	Recommendation
U-D1	Dart property-test library	glados vs hand-rolled testing/quick-style	UNVERIFIED — pin in Phase 1; the property obligation is fixed regardless
U-D2	Property-test seed policy	fixed seed (deterministic) vs rotating (broader)	fixed seed in CI (§11.4.50) + a separate explicit fuzz job; shrunk failures become committed fixtures
U-D3	Coverage floor start	70% vs 80% at Phase 1	70% ratcheting to 95% (overview §5.1) — avoids a green-but-shallow false start

10. The unit layer's place in the four-layer model (§11.4.108)

Unit is the **SOURCE layer only** — the cheapest, least conclusive of the four §11.4.108 layers. The discipline this imposes on every HelixVPN feature:

§11.4.108 layer	Unit's role	Who proves the rest
SOURCE	unit proves the logic compiles to the right result	—
ARTIFACT	unit does not prove the bytes landed in the build	post-build / INT
RUNTIME-on-clean-target	unit does not prove it is active on a deploy	E2E / SEC (e2e.md)
USER-VISIBLE	unit does not prove the user can use it	CHAL / REC (challenges.md)

A "green unit suite" is therefore necessary but **never sufficient** for a feature closure — the §11.4.108 forensic class (fixed-at-SOURCE, dead-at-RUNTIME) is exactly the failure a unit-only green hides. The coverage ledger ([overview §6](#)) enforces this mechanically: a feature's cell only reaches AUTONOMOUS_VERIFIED when its higher-layer cells carry §11.4.69 captured evidence, so a feature cannot ship on unit coverage alone.

Sources verified

- [Volume-8 overview — Testing, Acceptance & Anti-Bluff QA Strategy §1, §2 \(taxonomy row UNIT\), §5.1, §5.4 \(ab_run_n_times\), §6 \(coverage ledger\), §8 \(determinism/four-layer\)](#) — read 2026-06-26.

- Sibling component specs (cross-referenced for the unit surface): ../v03-control-plane/svc-policy.md, ../v03-control-plane/svc-ipam.md, ../v03-control-plane/svc-coordinator.md, ../v02-data-plane/transport-masque-quic.md, ../v02-data-plane/transport-selection-ladder.md, ../v02-data-plane/orchestrator-and-state.md, ../v04-client/state-management.md — filenames confirmed present 2026-06-26.
- Constitution clauses: §11.4.27 (no-fakes-beyond-unit / 100% test-type coverage), §11.4.5/.69/.107 (captured evidence / sink-side / liveness analyzer self-validation), §11.4.50 (determinism), §11.4.84 (mutation-residue quiescence), §11.4.108 (four-layer SOURCE→USER-VISIBLE), §11.4.132 (risk-order), §11.4.135 (regression guard), §11.4.1/§1.1 (FAIL-bluffs / paired mutation) — from CLAUDE.md in-context.
- External: RFC 9221 (HTTP/3 unreliable datagram) cited via **[research-masque]** in the overview Sources — **not independently re-fetched** for this spec doc (UNVERIFIED for exact wire-format constants; the framing round-trip obligation is format-agnostic).